

ServalCat 处理器设计报告

西北工业大学 3 队 指导教师：王党辉 王继禾

一、设计简介

ServalCat 是一个基于 LoongArch32 位精简版指令集^[1]的处理器软核，采用单发射顺序执行六级流水线结构。处理器实现了除浮点外全部 67 条整数及特权指令，支持例外与中断机制，实现了除浮点相关外全部例外，实现了除 EUEN 外所有控制状态寄存器。可启动 Linux 操作系统^[2]。

处理器包含分离的指令 Cache 和数据 Cache 各 8KiB。采用虚 Index 实 Tag 的方式访问，两路组相联设计，Cache 行大小为 16Byte。Cache 内部为三级流水结构。

处理器包含内存管理单元（MMU），TLB 分为两级，第一级 ITLB 和 DTLB 均使用 4 项直接映射结构，第二级混合 TLB 使用 16 项全相联结构，支持 4K 页和 4M 页。

处理器支持简单动态分支预测，预测失败代价为 3 周期。使用 256 项两位饱和计数器实现分支方向预测，64 项直接映射 BTB 实现分支地址预测。

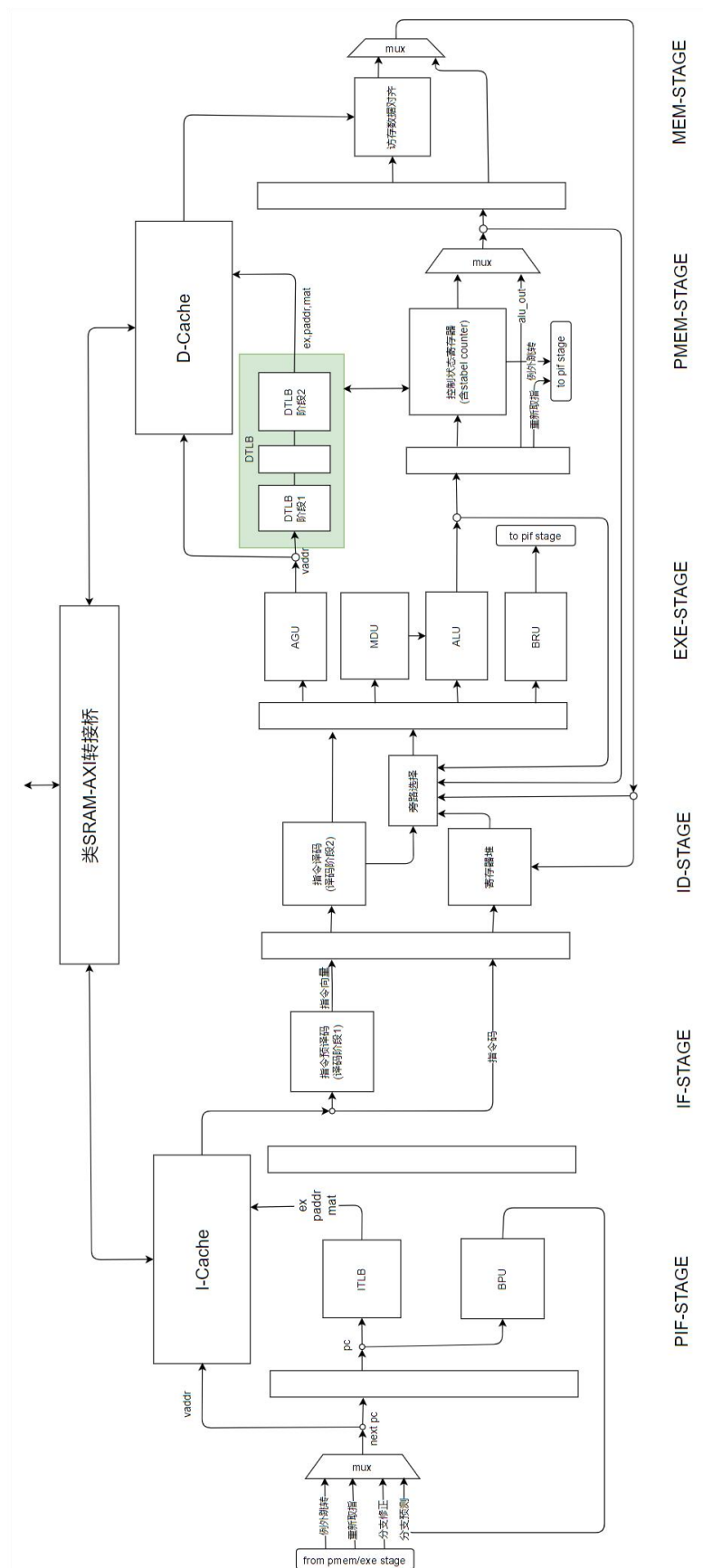
使用了多种优化方法提升频率，在性能测试的环境下处理器运行的最高频率为 145.45MHZ。

二、设计方案

（一）微架构简介

本设计采用单发射六级流水线架构，六个流水级分别称为预取指（PIF）、取指（IF）、译码（ID）、执行（EXE）、预访存（PMEM）、访存（MEM）。

经典五级流水线在加入 Cache 和 TLB 后，取指和访存均存在较为复杂的逻辑，会产生较长的路径，影响频率的提升。我们的设计希望最大程度地将取指和访存的逻辑平均分布在多个流水级，并挖掘其中的并行计算可能，以提升处理器运行频率，因而将 IF 和 MEM 级进行了拆分，具体的优化策略见第三部分。图 1 展示了我们处理器的微体系结构。



（二）乘除法器

1、乘法器

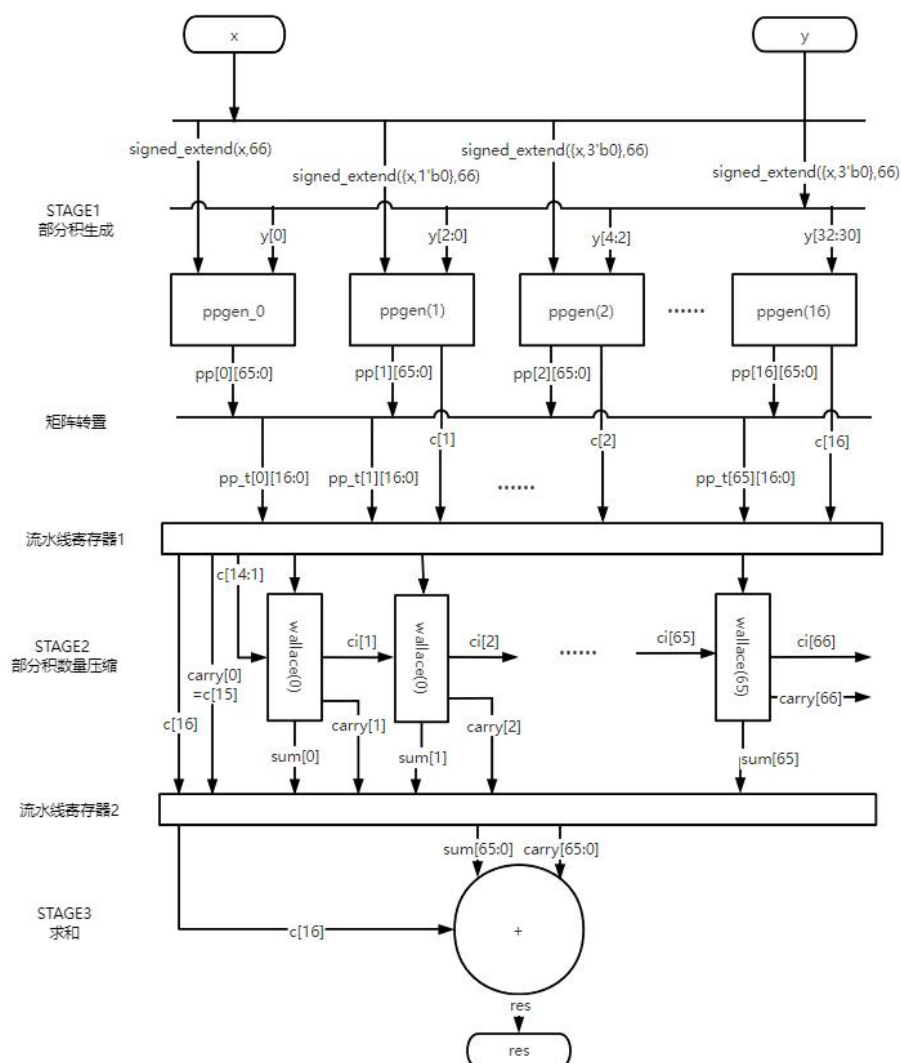


图 2 乘法器

如图 2 所示，乘法器支持两个 33 位数相乘，设计为流水线结构，分为三个流水级。第一级通过两位 booth 算法产生部分积，第二级通过华莱士树结构将 16 个部分积压缩为两个，第三级对最后两个部分积求和得到最终计算结果^[3]。

目前版本处理器暂时不支持乘法指令的流水化，计算过程中对执行级及之前的流水级进行阻塞，每一条乘法指令需阻塞两个时钟周期。

2、除法器

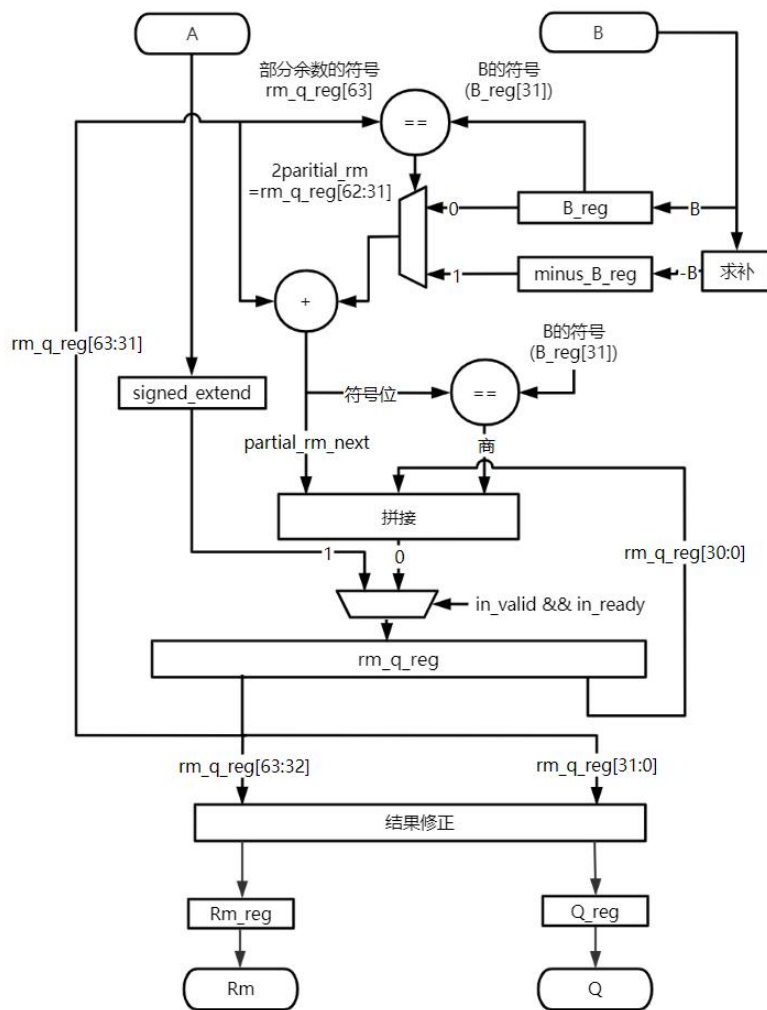


图3 有符号除法器

如图3所示，有符号除法器采用多周期设计，计算一次阻塞34个时钟周期。无符号除法器采用类似的方法，但无需考虑负数，设计更加简单，计算一次阻塞33个时钟周期。

（三）精确例外机制

精确例外机制的实现包含例外检查、例外报出和例外返回三部分。

PIF、ID、EXE、PMEM四级各负责检查出特定的一些例外，并将例外标记和例外编码随着指令沿着流水线向后传播。

在PMEM级访问控制状态寄存器时，处理器报出例外，对PMEM以及之前的流水级均进行flush，并跳转到例外处理子程序入口地址处继续执行。PMEM之后的流水级则不受影响。

例外返回时，由软件写入返回地址，以决定是否重新执行报例外的指令，执行ertn指令后跳回返回地址继续正常的执行流程。

（四）两级 TLB 设计

初赛时的混合 TLB 即使拆分为两级访问，依然对频率提升有阻碍，我们采用两级 TLB 机制来进一步进行优化。第一级将 ITLB 和 DTLB 分开，均采用 4 项直接映射设计。由于 DTLB 在 PMEM 级产生例外后就立刻要报出，控制信号的时序较为紧张，我们依然将 DTLB 的访问逻辑分布在 EXE 和 PMEM 两级。

设计中,ITLB 与 L2 TLB 同时访问,DTLB 的第二阶段与 L2 TLB 同时访问。L1 TLB miss 时先将 L2 TLB 的内容回填到 L1 TLB,再重新访问。在 L2 命中的情况下,ITLB 和 DTLB miss 的代价分别为 2 周期和 3 周期。

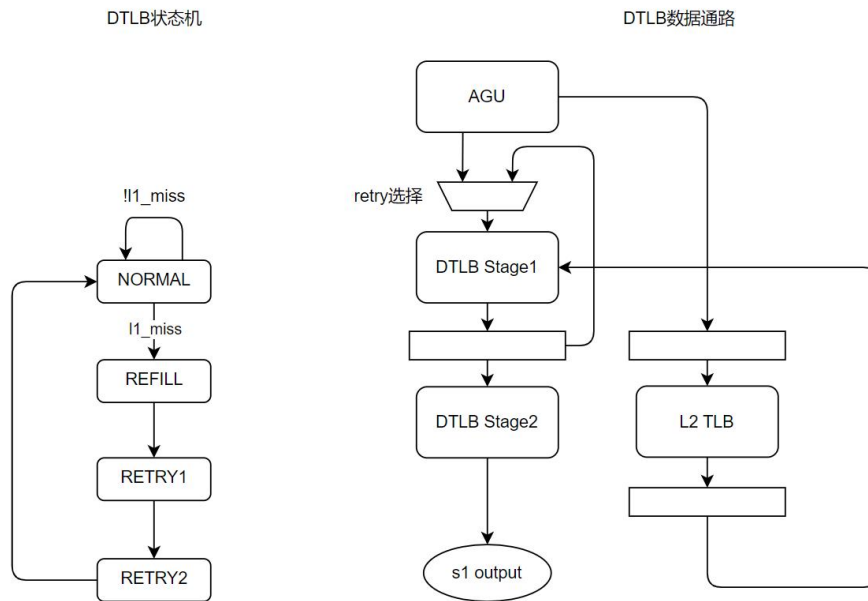


图4 DTLB数据通路和状态机

图 4 展示了 DTLB 的数据通路和状态机。L2 TLB 目前设置为 16 项，由于无需串接后续控制逻辑，加上采用重填重试的设计，访问 L2 时无需对奇偶页进行选择，L2 TLB 逻辑较为简单，可在一周期内完成访问。也可配置成更多项数，在不影响频率的前提下需花费更多周期来访问。

三、性能优化技术

（一）分支预测

本设计中，分支地址的计算在 EXE 流水级，错误预测的代价为 3 个时钟周期。

分支预测模块^[4]（Branch Predict Unit，简称 BPU）实现了整个分支预测的逻辑，包含方

向预测和地址预测。方向预测器包含由 256 个两位饱和计数器构成的模式历史表（Partten History Table，简称 PHT）。地址预测器包含一个 64 项直接映射结构的分支目标缓冲器（Branch Target Buffer，简称 BTB），可缓存除 JIRL 指令以外其他所有类型转移指令的地址。本设计不考虑 JIRL 指令的预测，固定产生 3 周期的执行代价。

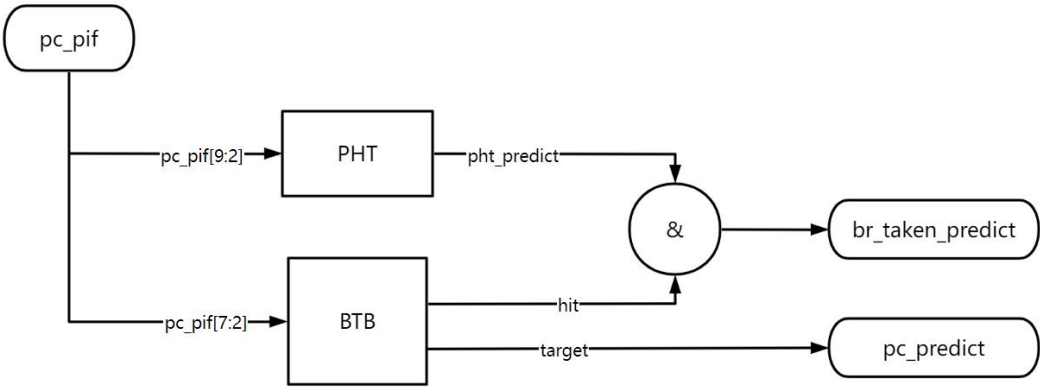


图5 BPU预测逻辑

图5展示了BPU的预测逻辑，以PIF级的PC（的一部分）作为地址同时访问PHT和BTB。当PHT预测为跳转，并且BTB命中时，指令最终被预测为跳转，并通过BTB得到跳转目标地址。PHT预测为非跳转或BTB miss时，指令预测为非跳转。

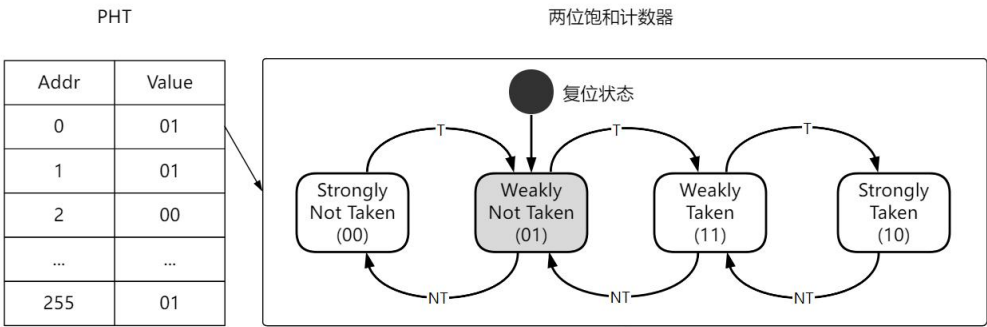


图6 模式历史表（PHT）

分支方向预测使用 PC 的一部分（9~2 位）寻址 PHT 实现。PHT 包含 256 个表项，每个表项为一个两位饱和计数器。

分支地址预测使用 64 项直接映射的 BTB。并且引入部分 Tag 比较机制。考虑到跨页跳转指令的稀少性，在判断 BTB 是否命中时，仅比较 4K 页的偏移量部分（除去最低 2 位和直接映射的 index，即比较 pc[11:8]），将页号的比较移动到 IF 流水级，缩短了关键路径，带来了频率提升。

根据统计，在十个性能测试中，除去 JIRL 指令，PHT 对分支方向的预测正确率的几何平

均值为 88.9%，考虑 BTB miss 后，这一值下降为 88.2%。BTB miss 造成的影响非常微弱。

（二）全同步设计

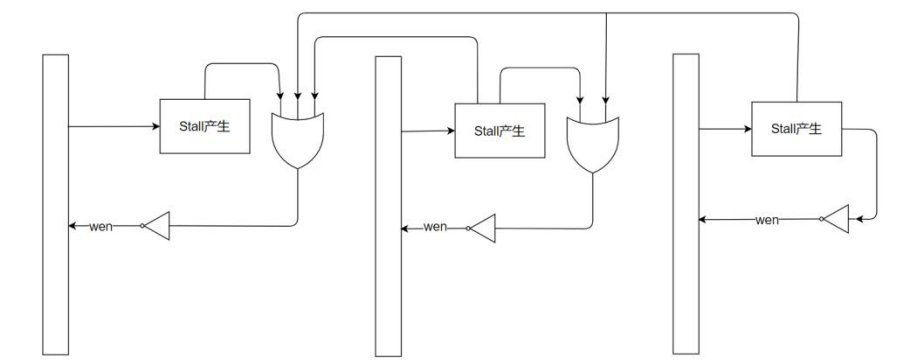


图 7 全同步设计的阻塞方法

处理器采用全同步设计，每一级产生的阻塞信号均能阻塞前面所有流水级。主流水线的流水级间不采用握手反压机制，以消除延迟较长的链式 allowin 信号，从而提升频率。

（三）流水级拆分与平衡

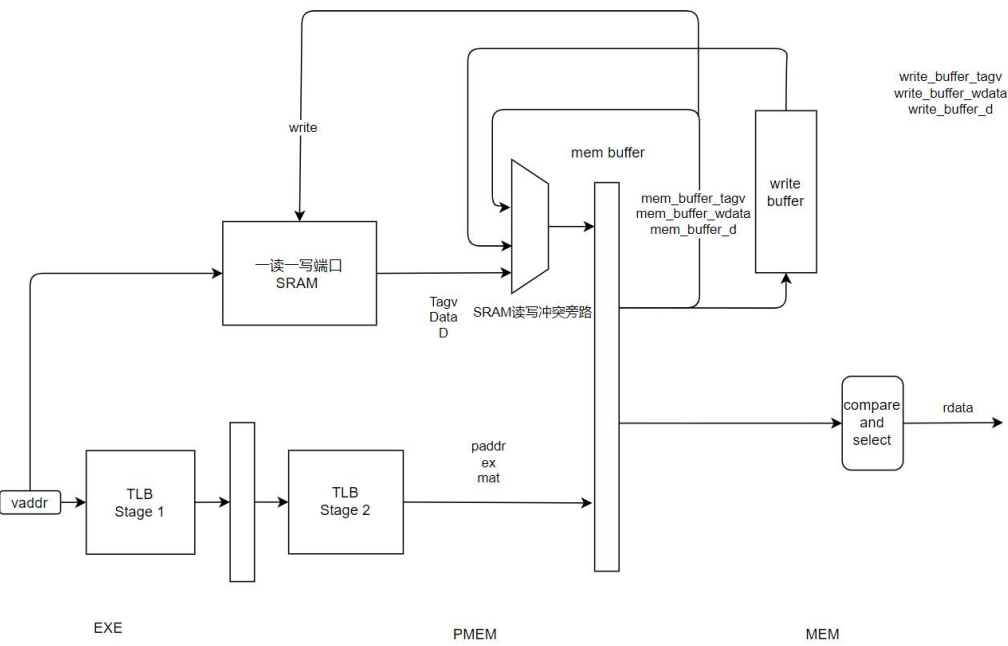


图 8 访存子系统 load/store 访存部分

如图 3-5，以访存子系统的 load/store 访存为例说明我们的优化方法。
我们在 EXE 级进行虚地址计算，并为 Cache 提供虚地址。PMEM 级同时完成虚实地址转换和 Cache SRAM 的读出，访问 TLB 结束后，主流水线为 Cache 提供实地址以及本次请求是

否有效等信息。最后，在 MEM 级完成 Cache 的 Tag 比较，数据选择和访存响应。

根据实际流水级时序压力，可以将一部分 Tag 比较逻辑并入 PMEM 级，在 PMEM 级和 MEM 级之间可以进行细化调整，以达到一个较为满意的平衡。

此外，为了解决 TLB 访问路径较长的问题，我们将后续参与产生复杂控制信号的 DTLB 访问流水化，将一部分逻辑并入 EXE 级，从而达到流水级间的平衡，这也是我们不使用 ALU 结果而单独使用 AGU 来计算虚地址的原因，后者可以省掉源操作数和结果选择的逻辑，达到用面积换时序的目的。

第二个例子，在图 1 中，我们展示了采用平衡流水级延迟对指令译码逻辑进行优化的策略，即利用 IF 级指令读出后剩余的时序资源提前做部分译码工作，来降低 ID 级的时序压力。

（四）组合逻辑优化

组合逻辑优化采用的方法主要为消除冗余逻辑、平衡各路延迟、面积换时序三种。

消除冗余逻辑除了用真值表分析的方法直接优化外，还可分析各个信号的所用是否必要，如各种有效信号只需要作用到最终修改处理器状态的数据上，而不需要无意义地作用到所有数据上。如果一个数据在某种条件下最终被多路选择器丢弃，则在该条件下数据的值任意改变均不会带来错误，这也带来了优化的空间。

在大赛提供的 FPGA 环境下，布线延迟通常显著高于逻辑延迟，平衡延迟和面积换时序的目标为减少关键路径上的逻辑级数。这主要通过挖掘并行计算的潜力来实现。

四、设计结果

（一）设计交付物说明

1、设计目录层次

-score_final.xlsx	决赛 Excel 表格，包含功能测试、性能测试得分的计算
-design.pdf	本设计报告
--sram_src/	目录，SRAM 目录，内部为空
--src/	目录，AXI 目录
--mycpu/	处理器 verilog 源代码
--tagv_dual_sram	设计中使用的 vivado IP 核
--data_dual_sram	设计中使用的 vivado IP 核
--perf_clk_pll.xci	性能测试的 clk_pll.xci
--bit/	
--func_test/	

		--soc_axi_func	目录内存放功能测试的 bit 流文件
		--soc_sram_func	内部为空，按照竞赛要求保持原状
		--perf_test	目录内存放性能测试的 bit 流文件
		--show/	系统展示目录
		--chip/	SoC 源代码目录
		--fpga/	vivado 工程文件目录
		--bit_and_bin/	系统展示的比特流文件和必要的二进制文件。
		--IP/	系统展示使用的 IP 核源码或.xci 文件，以及 CPU 核源码
		--src/	上板展示用到的程序的 c 语言源码

2、功能测试仿真、综合、上板步骤

- (1)下载大赛指定版本的功能测试发布包。
- (2)将 mycpu 文件夹复制到功能测试的根目录。
- (3)在 Vivado TCL console 中运行 create_project.tcl 脚本建立工程。
- (4)打开工程，分别手动添加 mycpu 文件夹内的源代码，以及两个 ip 核的 xci 文件。
- (5)如希望运行仿真，直接点击 Run Simulation 开始仿真即可。
- (6)如希望综合上板，运行 Generate Bitstream，等待综合、实现、生成比特流三个步骤完成，并将生成的比特流烧录到板子上。
- (7)上板时按复位按钮，即可观察到演示结果中的内容。

3、性能测试仿真、综合、上板步骤

- (1)下载大赛指定版本的性能测试发布包。
- (2)将 mycpu 文件夹复制到性能测试的根目录。
- (3)性能测试中已经包含 vivado 工程文件，不需要手动生成。将 perf_clk_pll.xci 文件改名为 clk_pll.xci，复制到./soc_axi_perf/rtl/xilinx_ip/clk_pll 目录下，替换其中的 clk_pll.xci。
- (4)打开工程，分别手动添加 mycpu 文件夹内的源代码，以及两个 ip 核的 xci 文件。
- (5)如希望运行仿真，先定制 axi_ram ip 核，选择想要跑的测试的 Coe File 来初始化内存，然后点击 Run Simulation 开始仿真即可。
- (6)如希望综合上板，运行 Generate Bitstream，等待综合、实现、生成比特流三个步骤完成，并将生成的比特流烧录到板子上。
- (7)上板时按复位按钮，然后按照性能测试的说明文档依次运行十个测试程序，即可观察到演示结果中的内容，计数会有微小的偏差。

4、展示内容上板以及复现的步骤

- (1)准备好显示器以及 VGA 线、网线、232 串口线。

(2)打开工程文件，开始综合和实现（intel core i7-12700h CPU 下 IP 的综合大约需要 25 分钟，设计的综合和实现大约 15 分钟），也可以跳过这一步骤直接使用提交包中提供的 SoC 比特流。

(3)如果实验箱 SPI flash 上没有烧入 LoongArch 架构的 PMON，则先按照 ChipLab 的说明烧入 PMON，其中 programmer_by_uart.bit 和 gzrom.bin 在提交包中有提供。

ChipLab 使用说明：https://chiplab.readthedocs.io/zh/latest/FPGA_run_linux/linux_run.html

(4)烧入第一步生成的比特流文件，按照 ChipLab 的说明启动 Linux 操作系统，**注意使用提交包中提供的 vmlinux 二进制文件**，直到进入 Shell。

(5)使用 ip addr add xxx.xxx.xxx.xx/24 dev eth0、ip link set eth0 up 搭建网络，tftp -g -r xxx.tar.gz xxx.xxx.xxx.xx 上传应用包（主机端将文件提前放在 tftp 服务器的目录下）。

(6)终端 2048 游戏：上传 my2048.tar.gz，解压后（tar -zxvf my2048.tar.gz 解压应用包），使用 ./my2048 运行程序，其中“w”，“s”，“a”，“d”分别控制上下左右，按动相应按键+回车，完成一次移动。除 2048 以外还提供了 maze 游戏。

(7)led 灯、8*8 点阵、数码管展示：上传 light.tar.gz，解压后，利用 chmod a+x light 对目标文件授权，使用 ./light 运行，此时开发板上的 led 灯、8*8 点阵、数码管成功出现展示效果。

(8)lcd 屏幕展示：上传 lcd.tar.gz，解压后，利用 chmod a+x lcd 对目标文件授权，运行 ./lcd，lcd 将显示一幅图片。

(9)vga 屏幕显示：首先将开发板与 vga 屏幕连接，此处 vga 显示模式采用 1440*900@60，然后上传 vga_select.tar.gz 和 vga_source.tar.gz，然后分别解压，此时按动 step0 按键启动 vga 屏幕，利用 chmod a+x vga_select 对目标文件授权，运行 ./vga_select，输入 mouse，屏幕中央显示 mouse 图片，输入 core，屏幕中央显示 core 图片。按动 step1 结束 vga 屏幕显示。

（二）设计演示结果

1、功能测试结果

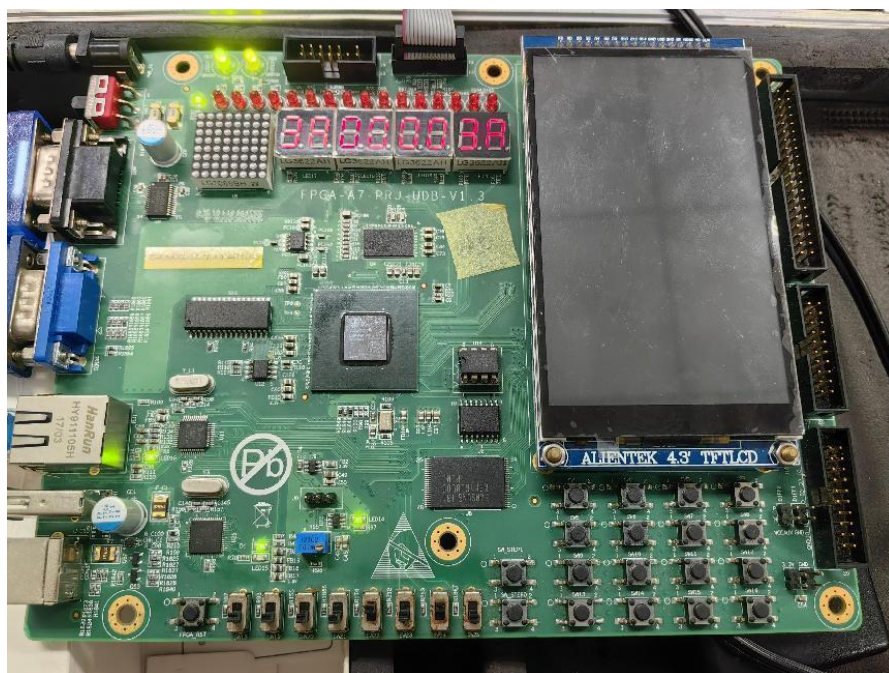


图 9 功能测试结果

如图 9 所示，设计通过了 AXI 接口所有 58 个测试点。

2、性能测试结果

序号	测试程序	myCPU		CPU count : SoC count	openla500	IPC _{mycpu} /IPC _{gs132}
		上板计时(16进制)			上板(16进制)	
		数码管显示 (CPU count) (最左开关拨下)	数码管显示 (SoC count) (最左开关拨上)		数码管显示 (CPU count)	
cpu_clk : sys_clk		145.45MHz : 100MHz		-	50MHz : 100MHz	-
1	bitcount	5d97c	405de	1.454055817	000572e5	0.931491877
2	bubble_sort	182753	109b60	1.454429266	00180f6a	0.996133123
3	coremark	506abe	374a49	1.454452977	0044469b	0.84902241
4	crc32	34f821	246abf	1.454520886	00331c68	0.964917542
5	dhystone	b83e0	7ead2	1.454433322	000945fd	0.805321895
6	quick_sort	1ecf4e	152f0e	1.454402572	0019fc53	0.843421737
7	select_sort	fcc32	adcec	1.454264267	000fd4b8	1.002107573
8	sha	1ddf95	148ace	1.454243404	001cfd8a	0.970457881
9	stream_copy	206dc	16510	1.453133205	0001bb30	0.854157256
10	stringsearch	18dd86	1118a4	1.454431374	0014dc07	0.838898387

性能分	IPC比值	0.903
	CPU频率	145.45MHz

图 10 性能测试结果

3、系统展示效果

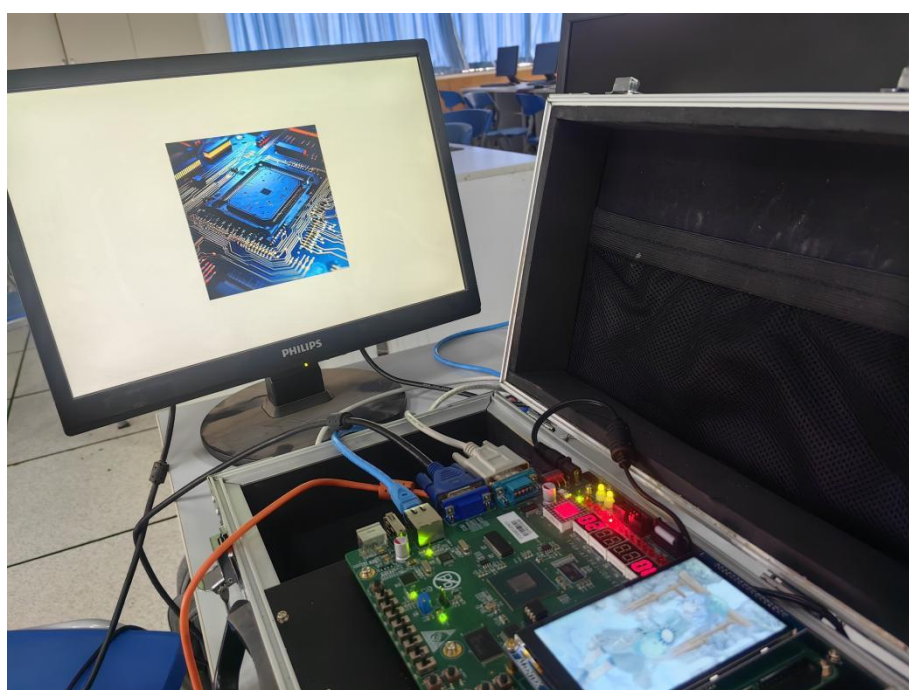
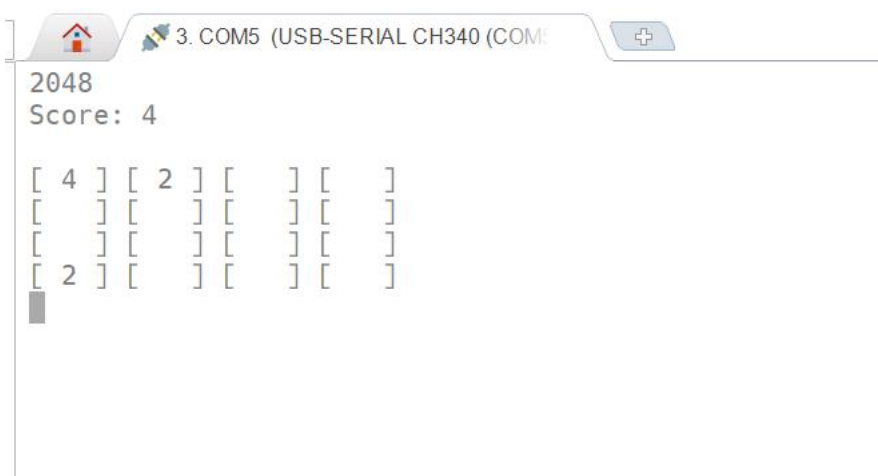
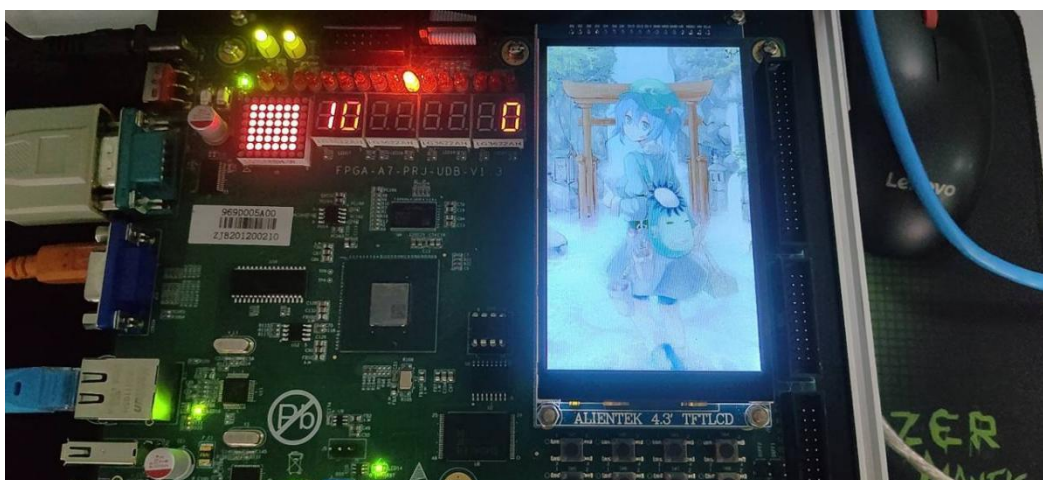


图 11 系统展示示意图

仅供参考，完整的展示项目见展示视频。

五、参考设计说明

（一）使用的第三方 IP

1、tagv_dual_sram

Vivado BlockMemoryGenerator 生成的伪双端口 BRAM，用于实现 Cache 的 Tag、V 联表。

2、data_dual_sram

Vivado BlockMemoryGenerator 生成的伪双端口 BRAM，用于实现 Cache 的 Data 表；

3、系统展示 SoC 说明

系统展示的 SoC 在 ChipLab 中 SoC 的基础上修改而来。

使用了 SoC 本身的源码和工程中包含的 vivado IP 核。

在原先 SoC 的基础上使用了几个新的 Vivado IP 核：

vga_sram

lcd_init_data

init_img_rom

均为 Vivado BlockMemoryGenerator 生成的 BRAM。

（二）参考设计

实现 IDLE 指令的 IDLE LOCK 机制参考了 OpenLA500 的设计。

流水级之间数据传递信号定义参考《CPU 设计实战》一书，全同步设计不需要 allowin 和 ready 信号，valid 和 bus 等信号依然保留。

TLB、Cache 的接口信号参考《CPU 设计实战》一书，内部设计不同。

AXI 接口的实现采用了先实现类 SRAM 接口，后实现类 SRAM-AXI 转接桥的方法，其中的技术参考《CPU 设计实战》一书。类 SRAM 总线接口信号的定义与书中的描述相同。

系统展示的 SoC 中 LCD 屏幕的驱动代码参考了往届作品 easter-mips^[7]。

六、参考文献

[1] 龙芯中科. 龙芯架构 32 位精简版参考手册 V1.03[EB/OL]. [2023-08-04]

[2] 龙芯教育. la32r-Linux[EB/OL]. [2023-08-04] <https://gitee.com/loongson-edu/la32r-Linux>.

- [3] 汪文祥,邢金章. CPU 设计实战[M]. 北京:机械工业出版社,2021.
- [4] 姚永斌. 超标量处理器设计[M]. 北京:清华大学出版社,2014.
- [5] Abel A. Automatic Generation of Models of Microarchitectures[D]. Saarland: Saarland University, 2020.
- [6] 龙芯教育. chiplab[EB/OL]. [2023-08-04]. <https://gitee.com/loongson-edu/chiplab>.
- [7] easter-mips. [EB/OL]. [2023-08-04]. <https://github.com/easter-mips/nscscc2020>.